

Section 1 RDFIA: report and summary

Titouan Guerin & Yacine Chettab

11 November 2025

Part I

ab - Introduction to Neural Networks

In this part, the central problem consists in designing a model capable of learning complex, non-linear relationships from labeled data to accurately predict the class of previously unseen examples.

Within this framework, we explored several fundamental aspects: the organization of datasets into training, validation, and test sets; the architecture of neural networks; the choice of activation functions; the selection of loss functions appropriate to the task (regression or classification); and the implementation of efficient optimization algorithms for adjusting model parameters.

To address these objectives, we adopted a systematic and analytical approach:

1. **Data understanding and handling:** We examined the roles of training, validation, and test sets and evaluated how the number of examples affects the model's generalization ability. This step highlighted the importance of data diversity and sufficient sample size.
2. **Network architecture design:** We implemented a simple feedforward network with an input layer, one or more hidden layers, and an output layer. Non-linear activation functions (ReLU, TanH) were added between linear layers to enable the network to learn complex relationships. Layer sizes were determined based on the number of input features, the number of classes to predict, and computational constraints.
3. **Loss functions and optimization:** We implemented key loss functions, including mean squared error for regression and cross-entropy for classification. Cross-entropy, combined with a SoftMax output, was preferred for its stable and effective gradient behavior during learning. Gradient descent algorithms (batch, mini-batch, and stochastic) were compared to assess their advantages and limitations, with mini-batch gradient descent emerging as the most practical compromise between stability and convergence speed.
4. **Backpropagation and gradient computation:** We implemented backpropagation to efficiently compute gradients of the loss with respect to network parameters, comparing its computational complexity with a naive approach. This exercise emphasized the importance of differentiable layers, memory management for intermediate activations, and awareness of potential gradient issues (vanishing or exploding gradients).
5. **Validation and interpretation:** The computed gradients were used to update network parameters, and model performance was evaluated on the test set. We also analyzed the influence of hyperparameters, such as learning rate and hidden layer size, on convergence behavior and predictive accuracy.

1 Theoretical foundation

1.1 Supervised Dataset

Questions

1. ★ **What are the train, val and test sets used for?**

The train set is used to train the model, the validation set is used to tune the hyperparameters during the training of the model, and the test set is used to evaluate the model's performance (data that is not in the train set, so not seen by the model previously).

2. What is the influence of the number of examples N ?

The number of examples N influences the model's ability to generalize, which means that by seeing enough examples, the model can learn to make predictions on unseen data. On the other hand, if N is too little, then the model will do more of a memorization of the training data, so it does not actually learn to generalize the problem. However, having too many samples N can be a problem in terms of training time and computational resources.

1.2 Network Architecture (forward)

Questions

3. Why is it important to add activation functions between linear transformations?

Having no activation functions between the linear layers would make the whole network linear (which is not what we are looking for). Chaining linear transformations is the same as having one linear layer but with just more weights. So the activation functions allow the network to learn non-linear patterns in the data.

4. ★ What are the sizes n_x , n_h , n_y in the figure 1? In practice, how are these sizes chosen?

- n_x is the size of the input layer, the number of features for one data point,
- n_h is the size of the hidden layer, a hyperparameter that is determined before training,
- n_y is the size of the output layer, the number of classes for classification problems.

5. What do the vectors $\hat{y}$ and y represent? What is the difference between these two quantities?

The vector $\hat{y}$ represents the predicted output of the model, while y represents the true labels.

6. Why use a SoftMax function as the output activation function?

The SoftMax function maps the output of the neural network to a probability distribution. This means that all the values outputted by the model are mapped between 0 and 1, and the sum of all the values are equal to 1: $\sum_{i=1}^n \hat{y}_i = 1$.

7. Write the mathematical equations allowing to perform the forward pass of the neural network, i.e. allowing to successively produce $\tilde{h}$, h , $\tilde{y}$ and $\hat{y}$ starting at x .

- $\tilde{h} = W_h x + b_h$
- $h = \text{ReLU}(\tilde{h})$
- $\tilde{y} = W_y h + b_y$
- $\hat{y} = \text{SoftMax}(\tilde{y})$

1.3 loss function

Questions

8. During training, we try to minimize the loss function. For cross entropy and squared error, how must the $\hat{y}_i$ vary to decrease the global loss function $\mathcal{L}$?

For the MSE, the prediction $\hat{y}$ is compared directly to y , so to minimize the loss, we want the difference between those two values to be as small as possible. In this case we work with real numerical values. This loss works for both regression and classification problems.

On the other hand, for the cross-entropy, we are working exclusively with probabilities, so to minimize the loss, we want the predicted probability $\hat{y}_i$ of the true class to be as close to 1 as possible, (and the predicted probabilities of the other classes to be as close to 0 as possible). Here y is just a probability vector, with just zeros except for the true class which is 1.

9. How are these functions better suited to classification or regression tasks?

MSE is naturally suited to regression, since its gradient is linear in the prediction error, providing a good measure of how far the prediction is from the target. However, in classification this linear behavior is less effective, because it does not properly penalize cases where the model is very confident but wrong (not enough at least).

Cross-entropy, on the other hand, has a gradient that grows rapidly when the predicted probability for the correct class is small. Basically you want to penalize a confident but wrong prediction because it impacts the other predicted probabilities. Since the penalty (or cost) of CE is logarithmic, and hence more harsh on wrong predictions, the gradient of the loss is larger and allows the model to learn faster.

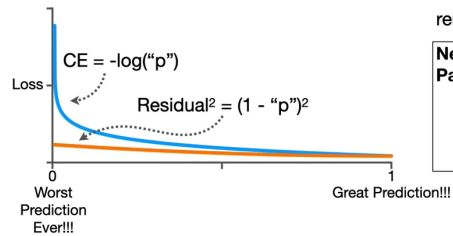


Figure 1: Cross-Entropy vs MSE derivative comparison.

1.4 Optimization algorithm

Questions

10. What seem to be the advantages and disadvantages of the various variants of gradient descent between the classic, mini-batch stochastic and online stochastic versions? Which one seems the most reasonable to use in the general case?

Method	Advantages	Disadvantages
Batch GD	Stable convergence, accurate gradients	High memory usage, slow & expensive for large datasets, possible local minimum
Mini-batch SGD	faster convergence, no local minima, better 'exploration' of the dataset	batch size tuning, noise
Online SGD	Fast updates, low memory usage	Noisy gradients, less stable convergence

Overall it is pretty safe to conclude that Mini-batch SGD is the most reasonable method to use in the general case, because it essentially is a mix of the two other methods: faster convergence and lower computational cost.

11. ★ What is the influence of the learning rate η on learning?

Recap: the learning rate η is a hyperparameter that controls how strongly the weights are updated during training.

A high learning rate means that the weights are updated strongly at each training step, which can lead to fast convergence but also to the overshooting of the optimal solution. Overshooting means that the model will diverge, or jump around the optimal solution without actually reaching it.

On the contrary, a low learning rate means that each update step changes the weights only slightly, which can lead to slow convergence. This can be helpful to explore the local minima, but can also lead to getting stuck there.

To address these drawbacks, one good technique is to change η over time. By starting with a high learning rate, and then decreasing it as we get closer to the optimal solution (over time), we allow the model to make bigger steps at the beginning of the training, and to then make smaller careful steps when we are close to the optimal solution.

12. ★ Compare the complexity (depending on the number of layers in the network) of calculating the gradients of the loss with respect to the parameters, using the naive approach and the backprop algorithm

For the naive approach, the gradient of the loss is calculated for each parameter independently. For example, for the weights of the second layer, we would calculate the gradient of the loss with respect to each weight w_{ij} in the weight matrix W_2 . And for the gradients of the first layer, we would re-calculate the gradients of the second layer, and then compute the

gradients with respect to the first layer. Even for a two-layer network, we can see that this approach is very computationally expensive, since we keep calculating the same gradients over and over.

With the backpropagation algorithm, each gradient is calculated only once. If we take our previous example with only two layers, after calculating the gradients of the second layer, we keep those gradients in memory and use them to calculate the gradients of the first layer. In essence, this means that all of the gradients are calculated in a single backward pass, without having to recompute the gradients of the loss like in the naive method.

This backpropagation algorithm is much more efficient and becomes mandatory for deep networks with many layers.

Remark on the $\mathcal{O}$ complexity:

For a network with L layers, let n_l be the number of parameters in layer l .

Let t_{avg} = the time to calculate the gradient for one parameter,

- computing $\frac{\partial \mathcal{L}}{\partial W_l}$ for each layer separately (naive):

$$\begin{aligned} T_1 &= t_{avg} \cdot \left(\sum_{l=1}^L \sum_{k=l}^L n_k \right) \\ T_1 &= t_{avg} \cdot n \left(\sum_{l=1}^L (L - l + 1) \right) \\ T_1 &= t_{avg} \cdot n(1 + 2 + \dots + L) = n \frac{L(L+1)}{2} \\ T_1 &= L^2 \cdot t_{avg} \cdot n_{avg} \sim \mathcal{O}(L^2) \end{aligned}$$

where n_{avg} is the average number of parameters per layer.

- For the backpropagation algorithm:

$$\begin{aligned} T_2 &= t_{avg} \cdot \left(\sum_{l=1}^L n_l \right) \\ T_2 &= t_{avg} \cdot (n + n + \dots + n) \\ T_2 &= t_{avg} \cdot L \cdot n_{avg} \sim \mathcal{O}(L) \end{aligned}$$

13. What criteria must the network architecture meet to allow such an optimization procedure ?

Each layer (linear, activation, cost ...) must be differentiable in order for the backpropagation to be possible. There also needs to be some form of memory to save every input and output of each layer during the forward pass as they are needed for the backward pass. Finally, the gradients produced must not be either extremely small or large, as they would stop the backpropagation (a.k.a the vanishing gradient problem) or the algorithm diverges (a.k.a the exploding gradient problem).

14. The function SoftMax and the loss of cross-entropy are often used together and their gradient is very simple. Show that the loss can be simplified by:

$$l = - \sum_i y_i \tilde{y}_i + \log \left(\sum_i e^{\tilde{y}_i} \right)$$

Recap: we write the cross-entropy loss as:

$$l = - \sum_i y_i \log(\hat{y}_i)$$

Here $\hat{y}_i$ is the output of the SoftMax function, so we can replace it in the loss:

$$\begin{aligned}
l &= - \sum_i y_i \log \left(\frac{e^{\tilde{y}_i}}{\sum_j e^{\tilde{y}_j}} \right) \\
l &= - \sum_i y_i \left(\log(e^{\tilde{y}_i}) - \log\left(\sum_j e^{\tilde{y}_j}\right) \right) \\
l &= - \sum_i y_i \left(\tilde{y}_i - \log\left(\sum_j e^{\tilde{y}_j}\right) \right) \\
l &= - \sum_i y_i \tilde{y}_i + \sum_i y_i \log\left(\sum_j e^{\tilde{y}_j}\right)
\end{aligned}$$

Since y is the label vector, composed of zeros except for the true class which is 1, then we know that the sum of all the y_i is equal to 1, so:

$$l = - \sum_i y_i \tilde{y}_i + \log\left(\sum_j e^{\tilde{y}_j}\right)$$

15. Write the gradient of the loss (cross-entropy) relative to the intermediate outputs $\tilde{y}$:

$$\frac{\partial l}{\partial \tilde{y}_i} = \dots \implies \nabla_{\tilde{y}} l = \begin{bmatrix} \frac{\partial l}{\partial \tilde{y}_1} \\ \vdots \\ \frac{\partial l}{\partial \tilde{y}_n} \end{bmatrix} = \dots$$

We are trying to find:

$$\frac{\partial l}{\partial \tilde{y}_i} = \sum_k \frac{\partial l}{\partial \hat{y}_k} \frac{\partial \hat{y}_k}{\partial \tilde{y}_i}$$

We need to compute $\frac{\partial l}{\partial \hat{y}_k}$:

$$\begin{aligned}
\frac{\partial l}{\partial \hat{y}_k} &= \frac{\partial}{\partial \hat{y}_k} \left(- \sum_k y_k \log(\hat{y}_k) \right) \\
&= -y_k \frac{\partial}{\partial \hat{y}_k} (\log(\hat{y}_k)) \\
&= -y_k \frac{1}{\hat{y}_k} \\
&= -\frac{y_k}{\hat{y}_k}
\end{aligned}$$

Next we compute $\frac{\partial \hat{y}_k}{\partial \tilde{y}_i}$, which is the derivative of the SoftMax function.

The derivative of the softmax has two forms:

$$\begin{aligned}
\frac{\partial \frac{e^{a_i}}{\sum_{k=1}^N e^{a_k}}}{\partial a_j} &= \frac{e^{a_i} \sum_{k=1}^N -e^{a_j} e^{a_i}}{\sum^2} = \frac{e^{a_i}}{\sum} \frac{\sum_{k=1}^N -e^{a_j}}{\sum} = S_i(1 - S_j) \quad \text{if } i = j \text{ and } \sum = \sum_{k=1}^N e^{a_k} \\
\frac{\partial \frac{e^{a_i}}{\sum_{k=1}^N e^{a_k}}}{\partial a_j} &= \frac{0 - e^{a_j} e^{a_i}}{\sum^2} = -\frac{e^{a_j}}{\sum} \frac{e^{a_i}}{\sum} = -S_j S_i \quad \text{if } i \neq j
\end{aligned}$$

So in our case, the derivative is:

$$\frac{\partial \hat{y}_i}{\partial \tilde{y}_i} = \hat{y}_i(1 - \hat{y}_i) \quad \text{and} \quad \frac{\partial \hat{y}_k}{\partial \tilde{y}_i} = -\hat{y}_i \hat{y}_k$$

So if we combine all of this:

$$\begin{aligned}
\frac{\partial \ell}{\partial \tilde{y}_i} &= \sum_{k=i} \left(-\frac{y_k}{\hat{y}_k} \cdot \hat{y}_i(1 - \hat{y}_k) \right) + \sum_{k \neq i} \left(-\frac{y_k}{\hat{y}_k} \cdot \hat{y}_i(0 - \hat{y}_k) \right) \\
&= \left(-y_i \frac{\hat{y}_i(1 - \hat{y}_i)}{\hat{y}_i} \right) + \sum_{k \neq i} \left(-y_k \frac{\hat{y}_i(-\hat{y}_k)}{\hat{y}_k} \right) \\
&= \left(-y_i(1 - \hat{y}_i) \right) + \sum_{k \neq i} (y_k \hat{y}_i) \\
&= -y_i(1 - \hat{y}_i) + \sum_{k \neq i} y_k \hat{y}_i \\
&= -y_i + y_i \hat{y}_i + \hat{y}_i \sum_{k \neq i} y_k \\
&= -y_i + y_i \hat{y}_i + \hat{y}_i(1 - y_i) \\
&= \hat{y}_i - y_i
\end{aligned} \tag{1}$$

In terms of matrices, the gradient of the loss than simply be expressed as:

$$\nabla_{\tilde{\mathbf{y}}} \ell = \begin{bmatrix} \frac{\partial \ell}{\partial \tilde{y}_1} \\ \vdots \\ \frac{\partial \ell}{\partial \tilde{y}_n} \end{bmatrix} = \hat{\mathbf{y}} - \mathbf{y}$$

Note: at the equation (1), we change the index from k to i for the term where k=i. This is because the sum only includes the case k=i, so k is effectively fixed to i. All other terms do not appear in this sum. This simplification allows us to replace k with i and remove the index from the expression.

16. **Using backpropagation, write the gradient of the loss with respect to the weights of the output layer $\mathbf{W}_y$. Note that computing this gradient uses $\nabla_{\tilde{\mathbf{y}}} \ell$. Do the same for $\mathbf{b}_y$.**

$$\frac{\partial \ell}{\partial W_{y,ij}} = \sum_k \frac{\partial \ell}{\partial \tilde{y}_k} \frac{\partial \tilde{y}_k}{\partial W_{y,ij}} = \dots \implies \nabla_{\mathbf{W}_y} \ell = \begin{bmatrix} \frac{\partial \ell}{\partial W_{y,11}} & \cdots & \frac{\partial \ell}{\partial W_{y,1n_h}} \\ \vdots & \ddots & \vdots \\ \frac{\partial \ell}{\partial W_{y,n_y 1}} & \cdots & \frac{\partial \ell}{\partial W_{y,n_y n_h}} \end{bmatrix} = \dots$$

So first of all,

$$\tilde{y} = \mathbf{W}_y \mathbf{h} + \mathbf{b}_y \quad \text{with} \quad \mathbf{h} \in \mathbb{R}^{n_h}$$

then by defining,

$$\tilde{y}_k = \sum_{j=1}^{n_h} W_{y,ij} h_j + b_{y,k},$$

We will now be able to simplify $\frac{\partial \tilde{y}_k}{\partial W_{y,ij}}$:

$$\frac{\partial \ell}{\partial W_{y,ij}} = \sum_{k=1}^{n_y} \frac{\partial \ell}{\partial \tilde{y}_k} \frac{\partial \tilde{y}_k}{\partial W_{y,ij}} = \frac{\partial \ell}{\partial \tilde{y}_i} h_j$$

In matrix form:

$$\nabla_{\mathbf{W}_y} \ell = \begin{bmatrix} \frac{\partial \ell}{\partial \tilde{y}_1} h_1 & \cdots & \frac{\partial \ell}{\partial \tilde{y}_1} h_{n_h} \\ \vdots & \ddots & \vdots \\ \frac{\partial \ell}{\partial \tilde{y}_{n_y}} h_1 & \cdots & \frac{\partial \ell}{\partial \tilde{y}_{n_y}} h_{n_h} \end{bmatrix} = (\nabla_{\tilde{\mathbf{y}}} \ell) \mathbf{h}^\top$$

This means that the gradient of the weights is obtained by multiplying the loss gradient with the output of the hidden layer.

For the bias b_y :

$$\frac{\partial \ell}{\partial b_{y,i}} = \sum_{k=1}^{n_y} \frac{\partial \ell}{\partial \tilde{y}_k} \frac{\partial \tilde{y}_k}{\partial b_{y,i}} = \frac{\partial \ell}{\partial \tilde{y}_i}$$

so in vector form

$$\nabla_{\mathbf{b}_y} \ell = \nabla_{\tilde{\mathbf{y}}} \ell.$$

17. ★ **Compute the gradients:** $\nabla_{\tilde{\mathbf{h}}} \ell, \nabla_{W_h} \ell, \nabla_{b_h} \ell$

(a) $\nabla_{\tilde{\mathbf{h}}_i} \ell$

First new define the deltas:

$$\delta_i^{(4)} = \frac{\partial \ell}{\partial \tilde{y}_i} \implies \delta_i^{(3)} = \frac{\partial \ell}{\partial W_y} = \sum_j \delta_j^{(4)} \frac{\partial \tilde{y}_j}{\partial W_y} = \sum_j \delta_j^{(4)} h_{ij}$$

and

$$\delta_i^{(2)} = \frac{\partial \ell}{\partial \tilde{h}_i} = \sum_j \delta_j^{(3)} \frac{\partial h_j}{\partial \tilde{h}_i} = \delta_i^{(3)} (1 - \tanh^2(\tilde{h}_i))$$

so $\delta_i^{(3)} (1 - \tanh^2(\tilde{h}_i)) = \nabla_{\tilde{\mathbf{h}}_i} \ell$

(b) $\nabla_{W_h} \ell$

We define:

$$\delta_i^{(1)} = \frac{\partial \ell}{\partial W_h} = \sum_j \delta_j^{(2)} \frac{\partial \tilde{h}_j}{\partial W_h} = \sum_j \delta_j^{(2)} x_{ij}$$

So $\nabla_{W_h} \ell = \sum_j \delta_j^{(2)} x_{ij}$

(c) $\nabla_{b_h} \ell$

And finally,

$$\frac{\partial \ell}{\partial b_h} = \sum_j \delta_j^{(2)} \frac{\partial \tilde{h}_j}{\partial b_h} = \sum_j \delta_j^{(2)}$$

So $\nabla_{b_h} \ell = \sum_j \delta_j^{(2)}$

Part II

cd - Convolutional Neural Networks

These practicals focused on the study and implementation of convolutional neural networks (CNNs) for image classification tasks. The goal was to understand core concepts such as convolutions, pooling, activation functions, and full network architectures, and to evaluate their ability to extract hierarchical representations from visual data. We implemented networks from scratch, experimented with training strategies including data normalization, augmentation, dropout, batch normalization, and learning rate scheduling, and assessed performance on datasets such as Fashion-MNIST, Kuzushiji-MNIST, and CIFAR-10. This approach combined theoretical understanding with practical experimentation to analyze factors affecting convergence, stability, and generalization of CNN models.

1 Introduction to convolutional networks

1. **Considering a single convolution filter of padding p , stride s and kernel size k , for an input of size $x \times y \times z$ what will be the output size ? How much weight is there to learn ? How much weight would it have taken to learn if a fully-connected layer were to produce an output of the same size ?**

For the output size, the formula for the output size of the convolution is:

$$Size_{output} = ([\frac{x + 2p - k}{s} + 1], [\frac{y + 2p - k}{s} + 1], N)$$

The number of channels z of an image does not influence the output size of the convolution (if we assume that the filter has the same number of channels as the image). In our case, N is equal to 1 since we only have one filter.

For the amount of weights N_w to learn:

$$N_w = k \times k \times z$$

If we were to use a fully-connected layer, we would need to connect each input neuron with each output neurons, thus the number of weights is:

$$N_{w_{fc}} = x \times y \times z \times [\frac{x + 2p - k}{s} + 1] \times [\frac{y + 2p - k}{s} + 1] \times N$$

2. **★ What are the advantages of convolution over fully-connected layers ? What is its main limit ?**

Using convolution layers over fully-connected layers is better for multiple reasons:

- A FC layers require at least one neuron per pixel per channel, whereas a convolution layer requires less weights for the filters since it uses the same filter for all of the pixels.
- A FC layer is less sensitive to small spatial variations (like translations or slight rotations) since filters can capture better local information.

Concerning their limitations, convolution filters extract information locally. While they do a good job at extracting patterns like edges and corners, they might not extract global information from the image as well (like relations between different areas in the image).

3. **★ Why do we use spatial pooling ?**

Spatial pooling is important to deal with spatial variations such as translation, rotation or zoom in/out. It enlarges the neighborhood of the considered local information around each pixel. By computing a simple value such as the average or max value (min value too although it is less popular) of each neighborhood, we can not only capture a better spatial information but also without any additional weights to learn.

Additionally, with the way that pooling layers are designed, we end up with a output of a lower size. Each convolution and pooling step reduces the spatial dimensions of the feature maps, which decreases computational cost and makes it easier to stack additional convolutional layers.

4. **★ Suppose we try to compute the output of a classical convolutional network (for example the one in Figure 2 VGG16) for an input image larger than the initially planned size (224×224 in the example). Can we (without modifying the image) use all or part of the layers of the network on this image ?**

The architecture illustrated in Figure 2 (e.g., VGG16) can be applied to input images of different sizes. In the early convolutional layers, the spatial dimensions of the image are progressively reduced, either by fixed-size filters or by pooling operations, but these operations remain valid regardless of the input size. Up to this point, the convolutional part of the network can handle variable input dimensions, since convolutions and pooling are local operations that do not depend on a fixed image size. However, at the end of the network, there is a flattening layer that converts the final feature maps into a single vector. When the input image size changes, the dimensions of this vector also change. Consequently, the fully connected layers that follow (which expect a fixed input size) cannot be used directly without modification. To adapt them, one common approach is to replace the fully connected layers with convolutional ones (for instance, using 1×1 convolutions). This converts the model into a fully convolutional network, allowing it to process images of arbitrary size while maintaining the same learned filters and structure.

5. **Supposing that we are only interested only in one, central, output pixel of a convolution, how can we modify the convolutional filter to make it equivalent to a fully-connected layer.**

If we are only interested in the output corresponding to a single central pixel, a convolution can be made equivalent to a FC layer by choosing a filter that spatially covers the entire input image. Formally, let $X \in \mathbb{R}^{C \times H \times W}$ denote the input tensor, where C is the number of channels and $H \times W$ its spatial resolution. A FC layer producing M outputs performs:

$$y = W \text{vec}(X) + b, \quad W \in \mathbb{R}^{M \times (CHW)}$$

This operation can be expressed as a convolution by using M filters of spatial size $H \times W$ and stride of 1. Hence, a convolution with a kernel spanning the entire input area becomes mathematically equivalent to a FC layer for that central pixel.

6. **There is a modification of convolution called “unshared convolution”. This maintains the “sparse connectivity” pattern of convolutions, but each input pixel has its own weights (no weight sharing). For an unshared convolution layer, with an input of shape (BCHW) and Q filters of size 5×5 , how many parameters (not including biases) would this layer have?**

An unshared convolution keeps the sparse local connectivity pattern of convolutions but removes the weight-sharing property. Each spatial position (h, w) thus has its own filter weights.

Given an input of shape (B, C, H, W) and Q filters of size 5×5 , the number of parameters (excluding biases) is:

$$P_{\text{unshared}} = Q \times C \times 5 \times 5 \times H' \times W',$$

where (H', W') are the spatial dimensions of the output feature maps. If we assume stride 1 and padding that preserves spatial dimensions ($H' = H, W' = W$), we obtain:

$$P_{\text{unshared}} = 25 \times Q \times C \times H \times W$$

7. **We call the receptive field of a neuron the set of pixels of the image on which the output of this neuron depends. What are the sizes of the receptive fields of the neurons of the first and second convolutional layers ? Can you imagine what happens to the deeper layers? How to interpret it?**

For convolutional layers with kernel size k_l , stride s_l , and assuming stride 1 for simplicity, the receptive field grows as follows:

$$r_l = r_{l-1} + (k_l - 1), \quad \text{with } r_0 = 1.$$

For instance, with two successive 3×3 convolutions, the first layer neurons have a receptive field of 3×3 , and the second layer neurons a receptive field of 5×5 :

$$r_1 = 3, \quad r_2 = 3 + (3 - 1) = 5.$$

In general, for variable strides, the recursive relations are:

$$\begin{aligned} r_l &= r_{l-1} + (k_l - 1) j_{l-1}, \\ j_l &= j_{l-1} \times s_l, \end{aligned}$$

where j_l denotes the spacing between adjacent receptive fields at layer l in input pixels.

Interpretation. In early layers, receptive fields are small and capture local features such as edges or textures. As depth increases, receptive fields grow through stacking of convolution and pooling operations, allowing deeper neurons to integrate increasingly global information about the image (object parts or full objects). This hierarchical expansion explains how convolutional networks achieve a balance between local sensitivity and translation invariance: deeper layers become more invariant to small spatial shifts but less precise in spatial localization.

2 Training from scratch of the model

2.1 Network Architecture

8. **For convolutions, we want to keep the same spatial dimensions at the output as at the input. What padding and stride values are needed?**

The stride must be equal to 1. Since all convolution layers have a filter of 5×5 , we need to add a padding of 2 for both directions.

9. **For max poolings, we want to reduce the spatial dimensions by a factor of 2. What padding and stride values are needed?**

To reduce the spatial dimensions by a factor of 2, we add no padding and we use a stride of 2 in both directions.

10. **★ For each layer, indicate the output size and the number of weights to learn. Comment on this repartition.**

Hypothesis The input is batch of 32×32 . For simplicity, we can assume that the batch size is 1. The padding is 2 for convolution layers and a stride of 1. For pooling layers, the stride is of 2 with no padding.

booktabs

Layer	Output dimension	Number of weights
conv1	$32 \times 32 \times 32$	$5 \times 5 \times 32 = 800$
pool1	$16 \times 16 \times 32$	0
conv2	$16 \times 16 \times 64$	$5 \times 5 \times 64 = 1600$
pool2	$8 \times 8 \times 64$	0
conv3	$8 \times 8 \times 64$	$5 \times 5 \times 64 = 1600$
pool3	$4 \times 4 \times 64$	0
fc4	$1 \times 1 \times 1000$	$1024 \times 1000 = 1024000$
fc5	$1 \times 1 \times 10$	$1000 \times 10 = 10000$

The convolution layers have a very low number of weights relatively to the fully connected layers, about 10^3 times less.

11. **What is the total number of weights to learn? Compare that to the number of examples.**

The total number of weight is 1 038 000 which is about 20 times larger.

12. **Compare the number of parameters to learn with that of the BoW and SVM approach.**

The total number of trainable parameters is 1 038 000 dominated by the fully-connected layers. By contrast, in a classical *Bag-of-Words* (BoW) combined with a SVM classifier, the number of learnable parameters is much smaller. The BoW representation itself is obtained through a fixed codebook (e.g., via k -means clustering or a pre-trained model), and only the SVM weights are learned during training. If the BoW vector has V dimensions as the vocabulary size and there are C output classes, the total number of learnable parameters is approximately:

$$P_{\text{BoW+SVM}} \approx V \times C.$$

Even for a large vocabulary (e.g., $V = 1000$) and $C = 10$ classes, this represents only about 10 000 parameters—several orders of magnitude fewer than the CNN.

Thus, the CNN involves a far richer and more flexible parametrization, allowing it to learn hierarchical feature representations directly from data, but at the cost of a much higher computational and data requirement compared to the BoW+SVM approach.

2.2 Network Learning

13. **Read and test the code provided. You can start the training with this command : `main (batch_size, lr, epochs, cuda = True)`**

See executed cell in the notebook, we did not make any changes for the question, only executed the code.

14. **★ In the provided code, what is the major difference between the way to calculate loss and accuracy in train and in test (other than the the difference in data)?**

In the case of a test epoch, we do not pass any optimizer to do a backward pass and update the weights:

```
model.eval() if optimizer is None else model.train()
```

The test epoch is also done on data points that were not seen during the training epochs.

15. **Modify the code to use the CIFAR-10 dataset and implement the architecture requested above. (the class is `datasets.CIFAR10`). Be careful to make enough epochs so that the model has finished converging.**

The network that we are using for the CIFAR-10 dataset in our ConvNet class (from question 10) is:

```
self.features = nn.Sequential(
    nn.Conv2d(in_channels, 32, kernel_size=5, stride=1, padding=2),
    nn.ReLU(),
    nn.MaxPool2d(2, 2),
    nn.Conv2d(32, 64, kernel_size=5, stride=1, padding=2),
    nn.ReLU(),
    nn.MaxPool2d(2, 2),
    nn.Conv2d(64, 64, kernel_size=5, stride=1, padding=2),
    nn.ReLU(),
    nn.MaxPool2d(2, 2)
)
```

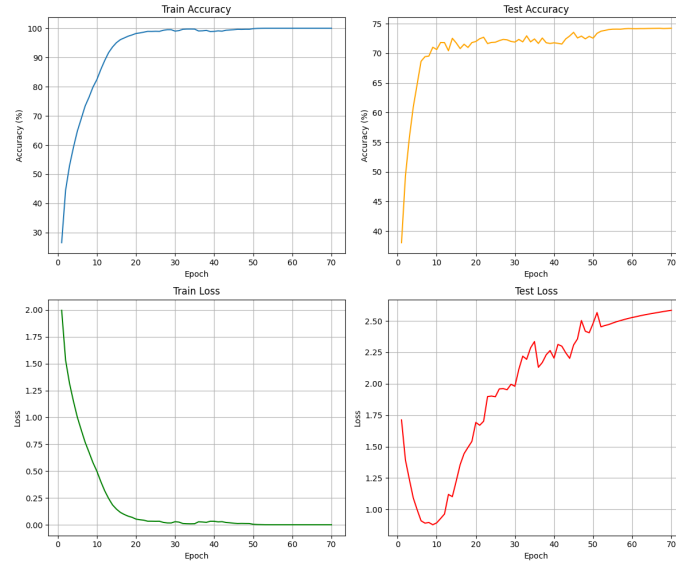


Figure 2: CIFAR-10 ConvNet train & test accuracy and loss across epochs

16. ★ **What are the effects of the learning rate and of the batch-size?**

The learning rate influences the importance of each update step. A high value means that each update is strong, which means that weights update faster (which is good in certain situations) but also leads to overshooting the optimal solution. This can often cause divergence or the model might not learn properly. A small learning rate does the opposite, where weights are updated very slowly. This makes learning more stable and more consistent, but with a drawback of slower. This can also lead the model to get stuck (converge) in a local optimum. The challenge here is to find a good value which allows the model to learn relatively fast, while also not overshooting. Usually, one good method is to start with a higher learning rate and decrease it over time to make the update steps more precise.

The batch size also affect the learning capabilities. Each batch estimates the gradient, having large batches allows a better estimation of it. However, this makes each training step much slower since we are dealing with larger quantities of data. This makes the training more stable but slower. On the other hand, smaller batches makes training faster but also makes the gradient estimation less accurate due to noise. Choosing the appropriate batch size mostly depends on the processing power available for the task.

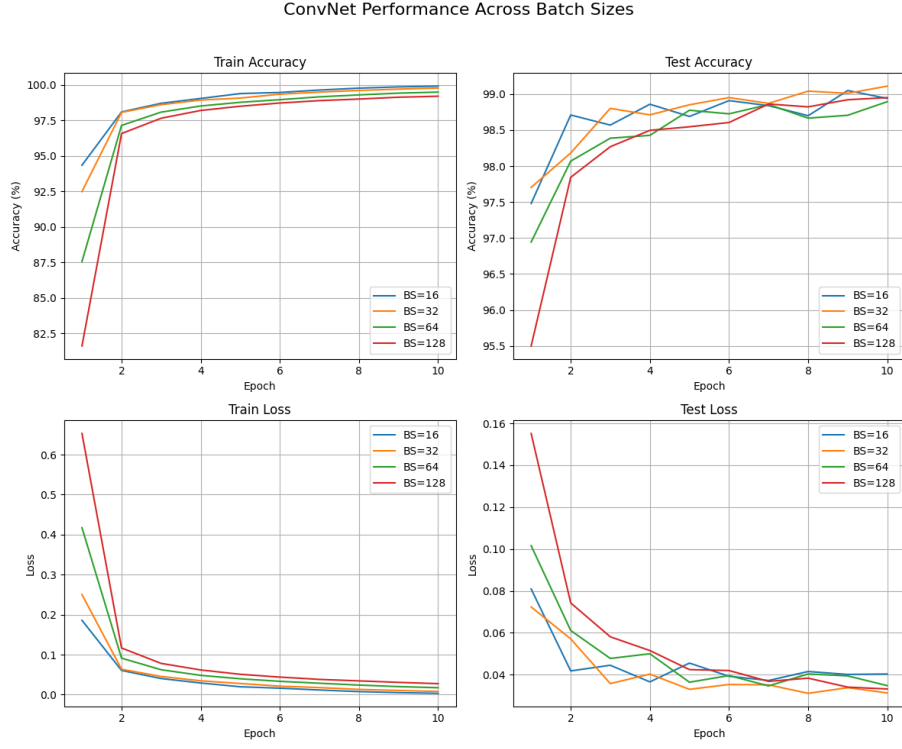


Figure 3: Influence of the batch size on accuracy and loss for training and testing on MNIST ($lr = 0.1$).

For the learning rate experiments, the batch size was kept at 32, based on the previous results suggesting it was a suitable choice.

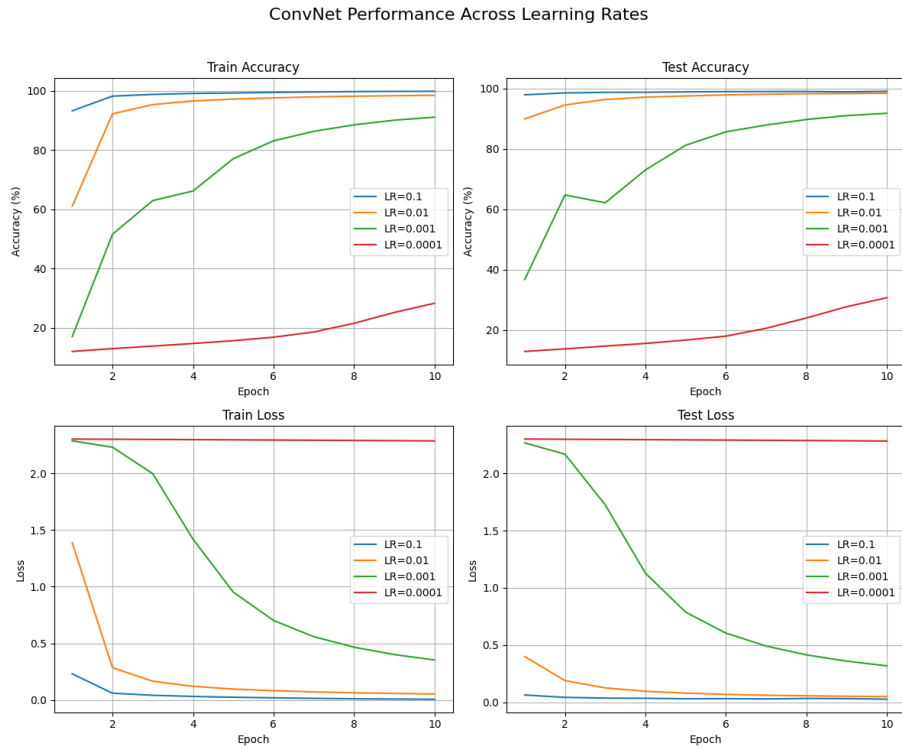


Figure 4: Influence of the learning rate on accuracy and loss for training and testing on MNIST ($bs = 32$).

17. **What is the error at the start of the first epoch, in train and test ? How can you interpret this ?**

To answer this question we look at our plots above, where the batch size is fixed, and we will study the blue curve which has a learning rate of 0.1 (this analysis is also valid for the orange curve since the two behave very similarly). We can observe around the first epoch that the test accuracy is higher than the train accuracy, and the test loss is lower than the train loss. This can happen because the test evaluation occurs after the model has already seen and learned from all the training batches of that epoch. At this point, the model parameters have already improved, so its performance on the test set—evaluated only once at the end of the epoch—can temporarily appear better than on the training data.

18. **★ Interpret the results. What’s wrong ? What is this phenomenon ?**

If we refer back to our figure 2, we can observe multiple things:

- (a) the train accuracy manages to reach a 100% accuracy around 30 epochs,
- (b) the test accuracy reaches a hard plateau of about 77% around 30 epochs,
- (c) the train loss converges to 0 at 30 epochs,
- (d) the test loss increases after the 7th epoch approximately, fluctuates a bit until the 30th epoch but with a general increase, and then keeps increasing.

These patterns clearly indicate that the model is overfitting: it learns the training data very well but fails to generalize to new examples. The early increase in test loss, even while training loss is decreasing, is a classic sign that the model starts memorizing noise or details specific to the training set instead of learning features that generalize. This phenomenon is likely due to the fact that the model is too powerful and has too many parameters for the task.

3 Results improvements

3.1 Standardization of examples

19. **Describe your experimental results.**

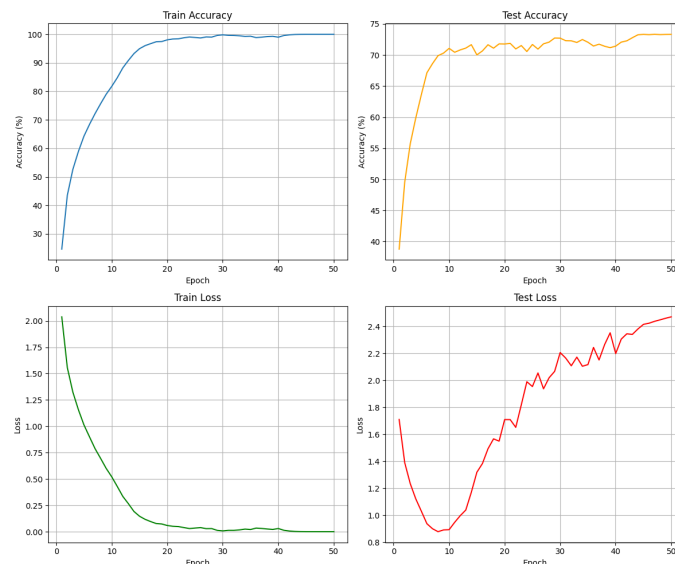


Figure 5: CIFAR-10 ConvNet train & test accuracy and loss across epochs

Compared to the results obtained in figure 2, the main difference here is that the model converges faster. Before we reached 100% accuracy at 50 epoch, whereas we reach it at

about 40 here. Concerning the test accuracy, they are about the same, between 73% and 75% in both cases. So this normalization does improve model training time.

20. **Why only calculate the average image on the training examples and normalize the validation examples with the same image?**

The mean and standard deviation are computed exclusively on the training examples because they represent the data distribution that the model learns from. Using statistics from the validation set would leak information about unseen data into the training process, leading to an optimistic and biased evaluation. By normalizing the validation and test examples with the same mean and standard deviation computed from the training set, we ensure that all datasets are expressed in a consistent feature space while preserving the integrity of the validation procedure. This practice guarantees a fair and unbiased assessment of the model's ability to generalize to unseen data.

21. **Bonus : There are other normalization schemes that can be more efficient like ZCA normalization. Try other methods, explain the differences and compare them to the one requested.**

- (a) *Standard normalization*: This was the technique used previously, so we will use it as a control normalization technique to compare with the other ones
- (b) *ZCA*: ZCA whitening decorrelates the pixel values of the images by applying a linear transformation derived from the covariance matrix of the dataset. This produces whitened inputs with reduced redundancy and more independent features, which can sometimes improve the model's ability to generalize. Nevertheless, this technique is computationally expensive and can introduce subtle distortions or artifacts in the data, which is why it is less commonly used in modern convolutional neural networks
- (c) *Per-image normalization*: In this approach, each image is normalized individually by removing its own mean and dividing by its own standard deviation. This helps eliminate lighting and contrast variations between images, making the network less sensitive to per-image intensity differences. On the other hand, it removes global brightness information and can sometimes lead to instability when absolute intensity patterns carry meaningful information for classification
- (d) *Min-max normalization*: Min-max normalization rescales each image's pixel values to a fixed range, typically between 0 and 1. This preserves the relative relationships between pixel intensities while ensuring consistent input bounds across the dataset, which can help stabilize learning. However, this technique is sensitive to outliers, as extreme pixel values can compress the range of the majority of pixels, potentially reducing contrast and informative variability

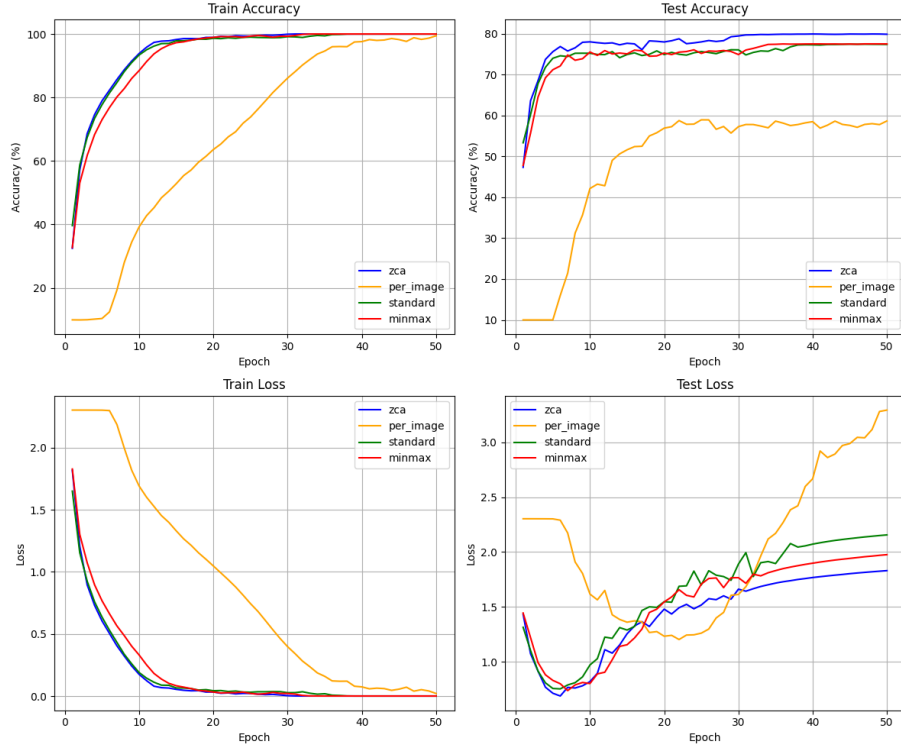


Figure 6: Train/Test accuracy and loss for various normalization techniques

If we look at the obtained curves, ZCA seems to obtain the highest accuracy in test by a couple points, which is a good improvement. It is worth mentioning that this improvement in accuracy required a lot more time to compute. The training with ZCA normalization took about two times longer than other techniques. To continue with, MinMax and our control normalization ended up getting the same results. We can also notice that the standard normalization performed a bit better than previously observed (see figure 5), though since each configuration was only run once, some minor variations are expected. This means that the reported scores should be interpreted as approximate indicators rather than statistically stable estimates of the true performance gap. Finally, the per-image normalization performed considerably worse than the others, most likely because normalizing each image independently destroys the relative contrast and global feature relationships that the model relies on to learn consistent patterns across the dataset.

3.2 Increase in the number of training examples by data increase

22. Describe your experimental results and compare them to previous results.

We applied data augmentation to the CIFAR-10 images to make the model more robust. This means each training image can appear slightly shifted or mirrored, helping the network learn features that generalize better instead of overfitting to exact pixel positions.

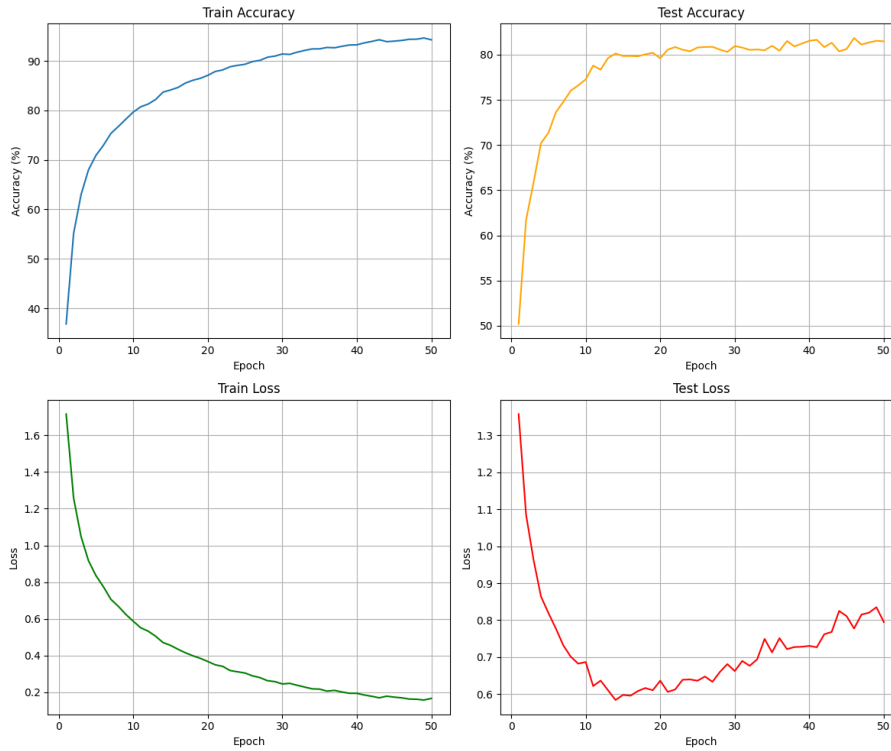


Figure 7: Train/test accuracy and loss for the data increase techniques

We can first observe on the train loss that the model did not reach 100% accuracy which means the model did not overfit to the data, so this is an improvement compared to previously tested methods. On top of that, the test accuracy also stayed high, about as much as during the normalization tests. Please note that we did keep the general data normalization (with the previously given mean and standard deviation). Below is an sample of the transformed images. We can see that some get flipped and/or get their shape changed.

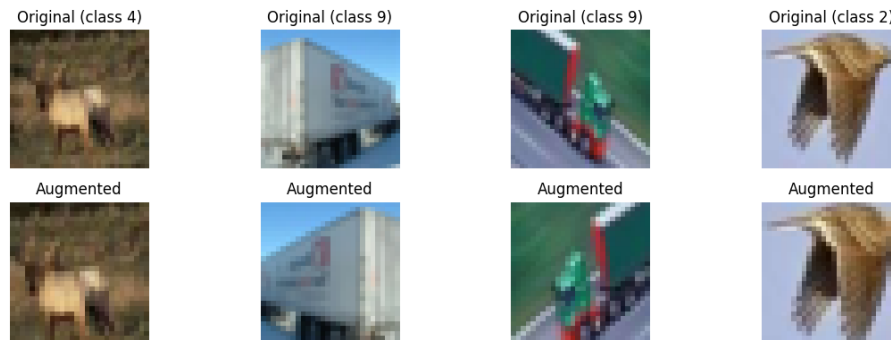


Figure 8: Randomly transformed images from the CIFAR-10 dataset (flip/crop)

23. Does this horizontal symmetry approach seems usable on all types of images? In what cases can it be or not be?

Horizontal flipping is suitable for datasets in which the left/right orientation does not affect the semantic meaning (like humans, animals, landscapes and certain objects). However, it is not suitable for cases like digit or character recognition (like MNIST and Kuzushiji-MNIST) because flipping the image could change certain classes that are not symmetrical. For example, if we flip the digit 2, we could end up with an image closer to the digit 9 or 5). Other cases could be found in medical images, satellite images or images with text recognition where flipping could either change the label or create meaningless features to the task. Thus, the horizontal symmetry's applicability depends on the task and on the dataset.

24. **What limits do you see in this type of data increase by transformation of the dataset?**

Since geometric transformations like cropping and flipping are simple, they have certain limitations:

- (a) The new images are simply a low-level transformations of the existing images, therefore these operations do not introduce fundamentally new features on the new images.
- (b) These transformations are rare in the scale of common geometric transformations that we may find in images of the real world, such as rotations, translations, scaling, and brightness.
- (c) Since the augmentation is done dynamically during training, validation and testing, we may not be able to apply it only on a certain number of

25. **Bonus : Other data augmentation methods are possible. Find out which ones and test some.**

Now we tested two new data augmentation methods:

- (a) Color and brightness transformations: random modifications of hue, contrast, saturation or brightness. The idea is that by exposing the network to variations in lighting and color, it may learn more robust features and improve generalization.
- (b) Simple geometric transformations: we applied small rotations, translations, and scaling to the images, which encourages the model to recognize objects even if they appear in slightly different positions or sizes, again aiming to improve generalization.

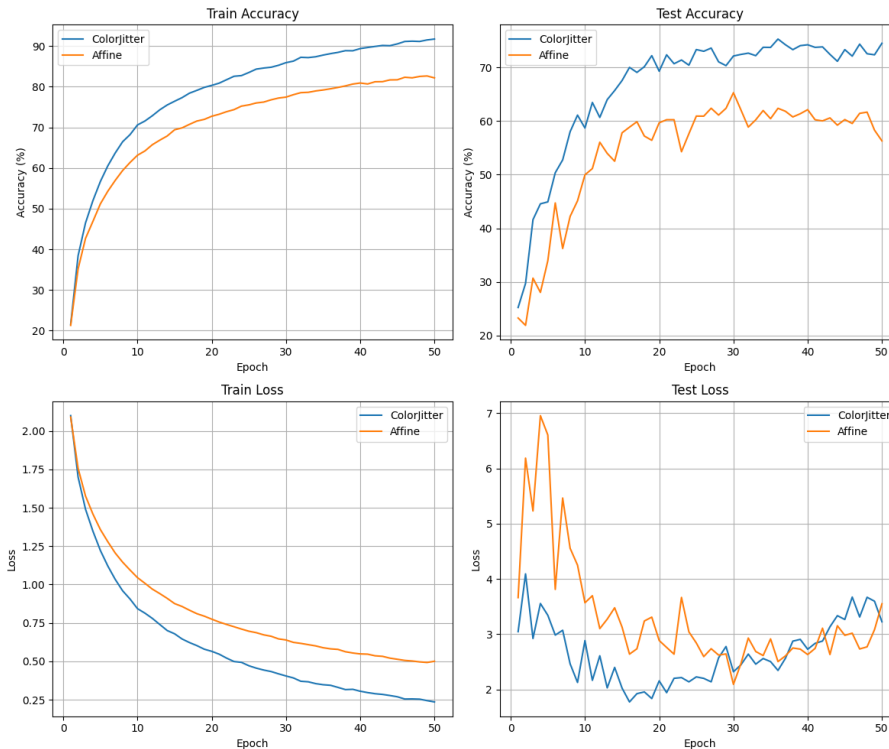


Figure 9: Train/test performance for two data augmentation techniques

Although both models were trained under the same conditions, the network trained with *RandomAffine* performed worse than the one with *ColorJitter*. This is likely because affine transformations introduce geometric distortions that may sometimes move or stretch important object features, making them harder for the network to recognize. In contrast, color and brightness variations preserve the spatial structure of objects while increasing data diversity, which can improve generalization without confusing the model.

3.3 Variants on the optimization algorithm

26. Describe your experimental results and compare them to previous results, including learning stability.

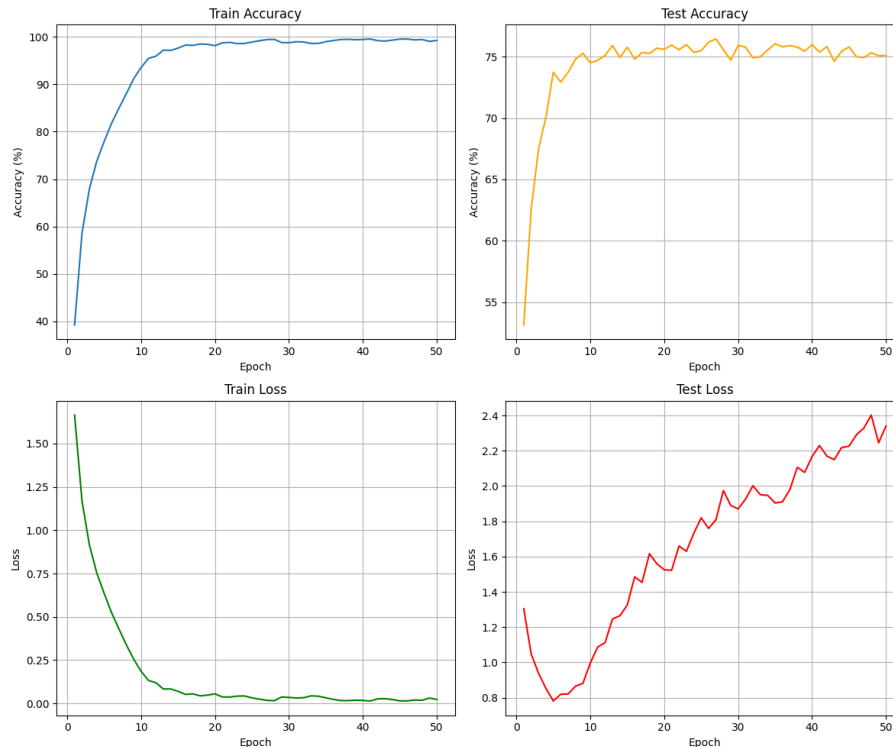


Figure 10: Train/test accuracy and loss with a learning rate scheduler (exponential decay = 0.95)

If we compare our results with the ones obtain in figure 5, which is the same setup but without the learning rate scheduler, the overall peak accuracy in test and train is reached earlier. However we can also observe that over time, the curves in figure 10 do not look as smooth as their counterparts. This can be explained by the fact that the learning rate scheduler introduces a change in the learning rate value throughout training. As the learning rate decays slowly, the optimizer continues to make non-uniform updates to the model parameters, which can lead to small oscillations around the area of convergence. Visually, the loss and accuracy curves appear less smooth, despite the improved convergence speed and overall performance (the accuracy in test is above 75%).

27. **Why does this method improve learning?**

Using an exponentially decaying learning rate improves learning stability and convergence by progressively reducing the step size of parameter updates as training progresses. At the beginning of training, a high learning rate allows the optimizer to make large updates and explore the parameter space efficiently, avoiding poor local minima. As the model approaches a region close to the optimum, smaller learning rates prevent oscillations and allow for finer convergence around the minimum. Thus, a dynamic adjustment of the learning rate provides a balance between exploration (fast learning initially) and exploitation (the fine and precision convergence), resulting in more stable loss curve compared to a constant learning rate.

28. **Bonus : Many other variants of SGD exist and many learning rate planning strategies exist. Which ones? Test some of them**

Besides SGD with an exponential decay of LR, numerous variants exist to improve convergence speed and robustness such as:

- **SGD with Momentum:** The momentum method introduces an exponential memory of past gradients to accelerate learning along consistent directions and reduce oscillations

in narrow valleys of the loss function. The update rule is:

$$v_t = \mu v_{t-1} - \eta \nabla_{\theta} \mathcal{L}(\theta_t), \quad \theta_{t+1} = \theta_t + v_t$$

where $\mu \in [0, 1)$ is the momentum coefficient, η the learning rate, and v_t the velocity term.

- Adagrad: Adagrad (Adaptive Gradient) adapts the learning rate individually for each parameter according to the cumulative magnitude of its past gradients. It scales down the learning rate for parameters with large updates, which is effective for sparse data:

$$r_t = r_{t-1} + (\nabla_{\theta} \mathcal{L}(\theta_t))^2, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{r_t + \epsilon}} \nabla_{\theta} \mathcal{L}(\theta_t)$$

- RMSProp: RMSProp (Root Mean Square Propagation) improves upon Adagrad by maintaining an exponential moving average of squared gradients rather than their full sum, preventing the learning rate from decaying too quickly:

$$r_t = \rho r_{t-1} + (1 - \rho)(\nabla_{\theta} \mathcal{L}(\theta_t))^2, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{r_t + \epsilon}} \nabla_{\theta} \mathcal{L}(\theta_t)$$

- Adam: Adam (Adaptive Moment Estimation) combines the benefits of Momentum and RMSProp by maintaining exponentially decaying averages of both the first moment (mean) and the second moment (uncentered variance) of the gradients:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}(\theta_t) \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L}(\theta_t))^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \\ \theta_{t+1} &= \theta_t - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} \end{aligned}$$

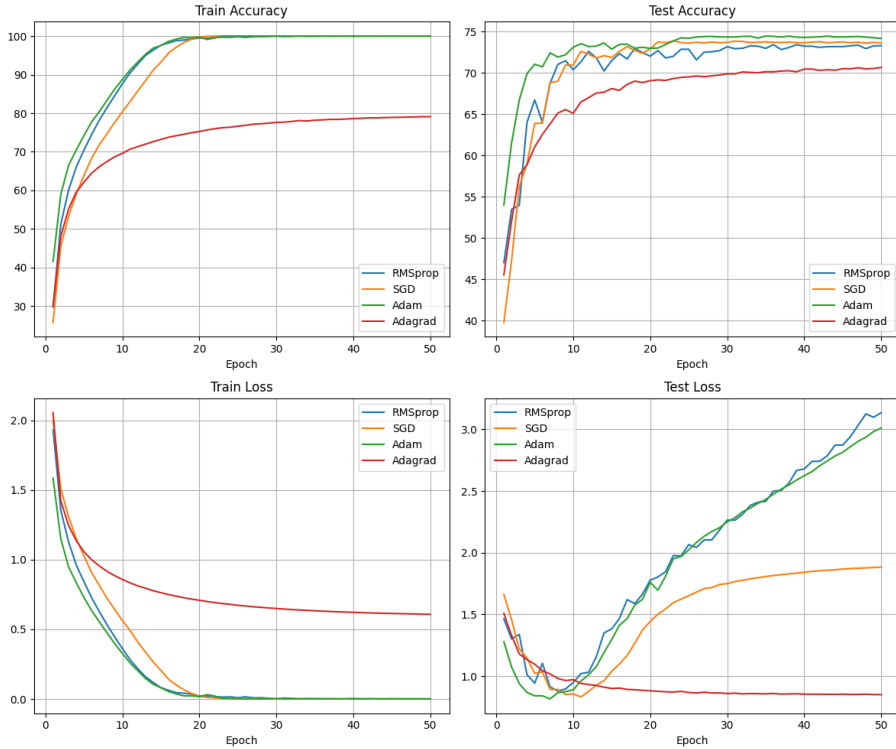


Figure 11: Train/test accuracy and loss curves for various optimizers

To comment on the results, we would like to note that each optimizer was initialized with its most commonly used learning rate. For instance, *AdaGrad's* default learning rate is 0.01

whereas *Adam's* is 0.001. This means that they are all configured slightly differently. On top of that, these curves were obtained only on a single training trial, so they do not represent the average performance of each method.

Concerning the accuracies, we can observe that the only optimizer that did not overfit is *Adagrad*. It 'only' achieved at 80% accuracy during train, but managed to get a very similar test score as the other optimizers (only a couple % points difference). This suggests that *Adagrad's* managed to generalize better, even if it converged more slowly. Additionally, we can see that the other 3 methods reached higher training accuracy quickly but did overfit as mentioned previously. Overall, these results emphasize that the choice of optimizer can significantly impact not just convergence speed, but also model robustness on unseen data, but also requires fine-tuning on the hyper-parameters of the optimizers.

3.4 Regularization of the network by dropout

29. Describe your experimental results and compare them to previous results.

We decided to test various dropout p values using vanilla *SGD* with a learning rate of 0.01. We tested three different values for p : 0.25, 0.5 and 0.75. These values were selected to answer the next questions of this section.

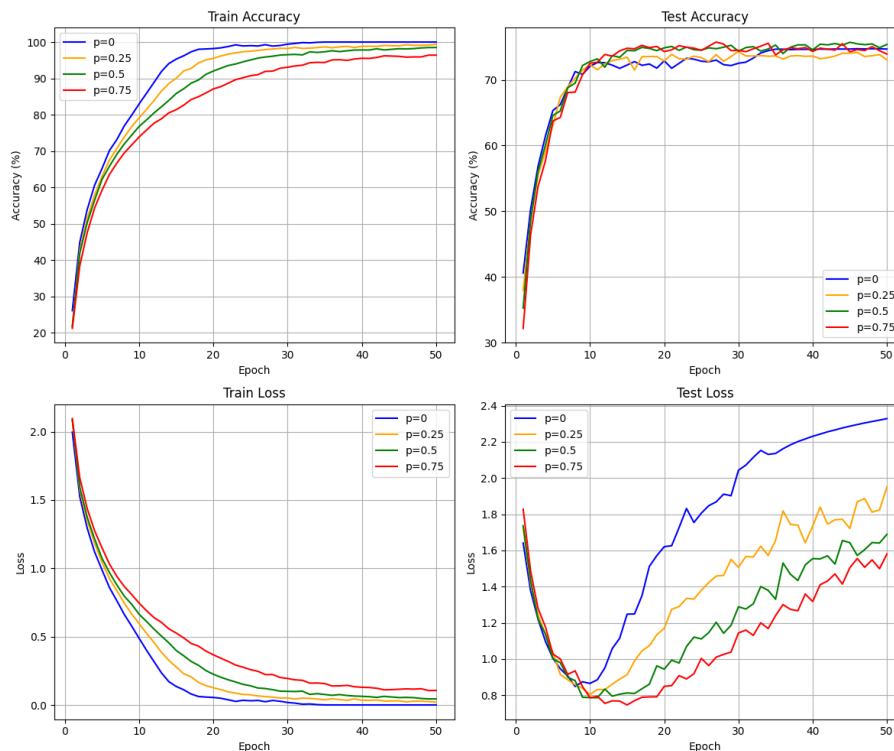


Figure 12: Train/test accuracy and loss for various p values of the Dropout layer

We can observe that when $p = 0$ (the vanilla model) and $p = 0.25$ (very close to vanilla), the model overfits. We can see that the other managed to be around 95% and 97% which is still a slight improvement. Concerning the test accuracy, the average and default value performed best by a little bit. We can assume that if we added more methods, like a learning rate scheduler then we could possibly see a bigger gap in performance between the curves. This will be tested in the last question 4.

30. What is regularization in general?

Regularization is a technique used to prevent overfitting by constraining the model's complexity. It penalizes overly flexible models that fit very well the training data but fail to generalize on the validation or/and test dataset. Common regularization methods include L_1 and L_2 weight penalties, early stopping, data augmentation, and dropout. Formally, in

L_2 regularization, the loss function is modified as:

$$\mathcal{L}'(\theta) = \mathcal{L}(\theta) + \lambda \|\theta\|_2^2$$

where λ controls the strength of the penalty term.

31. Research and "discuss" possible interpretations of the effect of dropout on the behavior of a network using it?

Dropout can be interpreted as an ensemble method. During training, each mini-batch uses a different "thinned" version of the network, where neurons are randomly dropped with probability p . At test time, all neurons are active, and their outputs are scaled by $(1 - p)$ to maintain the expected activation. This process effectively trains multiple sub-networks and averages their predictions, thus reducing variance and improving robustness. Mathematically, if x is the input and M is a mask matrix ($M_i \sim \text{Bernoulli}(1 - p)$ of the same size as x), the layer output is:

$$y = W \cdot (M \odot x)$$

32. What is the influence of the hyperparameter of this layer?

The dropout rate p determines the proportion of neurons randomly deactivated at each iteration.

- A small p (e.g., 0.1–0.3) has limited effect and may not sufficiently prevent overfitting.
- A large p (e.g., 0.6–1) may induce underfitting, as too many neurons are deactivated.

Empirically, values around $p = 0.5$ are common for fully connected layers, while smaller rates ($p = 0.25$) are used for convolutional layers. This hyperparameter should be tuned to balance between bias and variance.

33. What is the difference in behavior of the dropout layer between training and test?

Dropout behaves differently during training and testing:

- Training: Each neuron is dropped with probability p , forcing the model to learn redundant and distributed representations.
- Testing: All neurons are active, but their outputs are scaled by $(1 - p)$ to match the expected activations during training. This scaling ensures consistency between the average neuron activation during training and inference.

3.5 Use of batch normalization

34. Describe your experimental results and compare them to previous results.

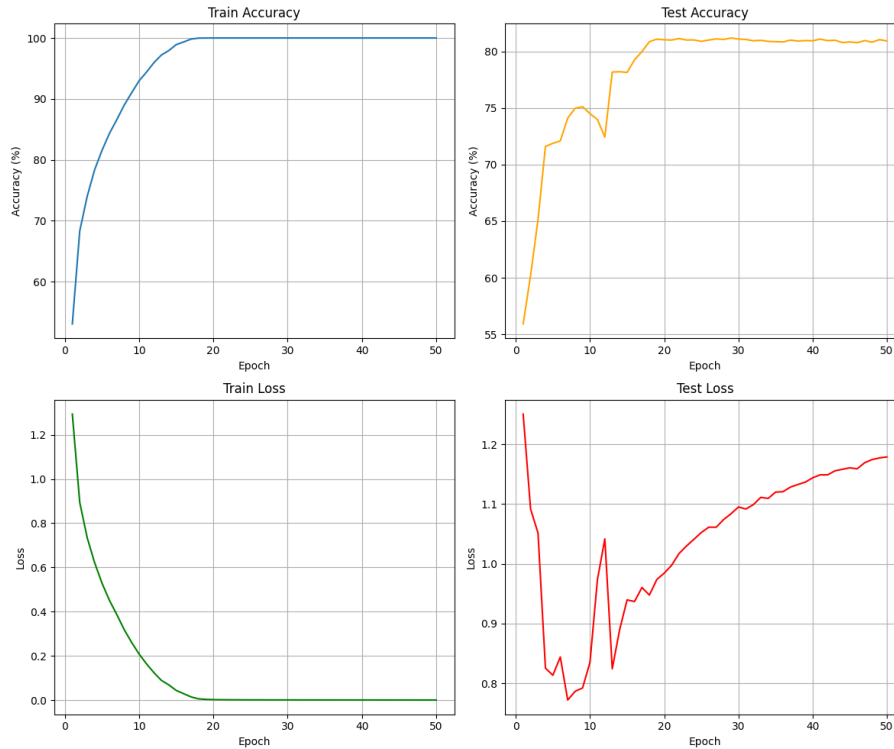


Figure 13: Train/test accuracy and loss when adding batch norm to ConvNet

The use of batch normalization after the convolution layers allows the network to be more stable and to learn faster. By ensuring that the batches have a stable distribution, the network becomes more robust to changes, thus enhancing the learning process. For figure 13, we only applied the batch normalization layer. We wanted to test it on the vanilla network to see its raw influence (no other techniques playing a role in the potential increase in accuracy). We can observe, compared to figure 2 that we get a notable increase in the test accuracy. The training accuracy shows that the model did overfit but that was to be expected. In the next section, we will train multiple models with added features to help fight overfitting while keeping a high test accuracy.

4 Bonus Conclusion: Incremental Model Enhancements

In this section, we train several models, progressively adding different techniques to enhance their performance. Although this section was not included in the PDF, we decided that combining all of our experiments into a series of models provides a clear conclusion to our work on CNNs. The following models will be trained:

1. **Model 0:** Vanilla + Adam ($\text{lr} = 0.001$)
2. **Model 1:** Model 0 + Lr Exponential Decay
3. **Model 2:** Model 1 + Standardization
4. **Model 3:** Model 2 + RandomCrop/Flip
5. **Model 4:** Model 3 + Batch Normalization
6. **Model 5:** Model 4 + Dropout ($p = 0.5$)

In our experiments, we started from a baseline model and incrementally added different techniques to observe their impact on training and performance. We decided to reuse the best methods we implemented throughout the experiments, which included a learning rate scheduler, image

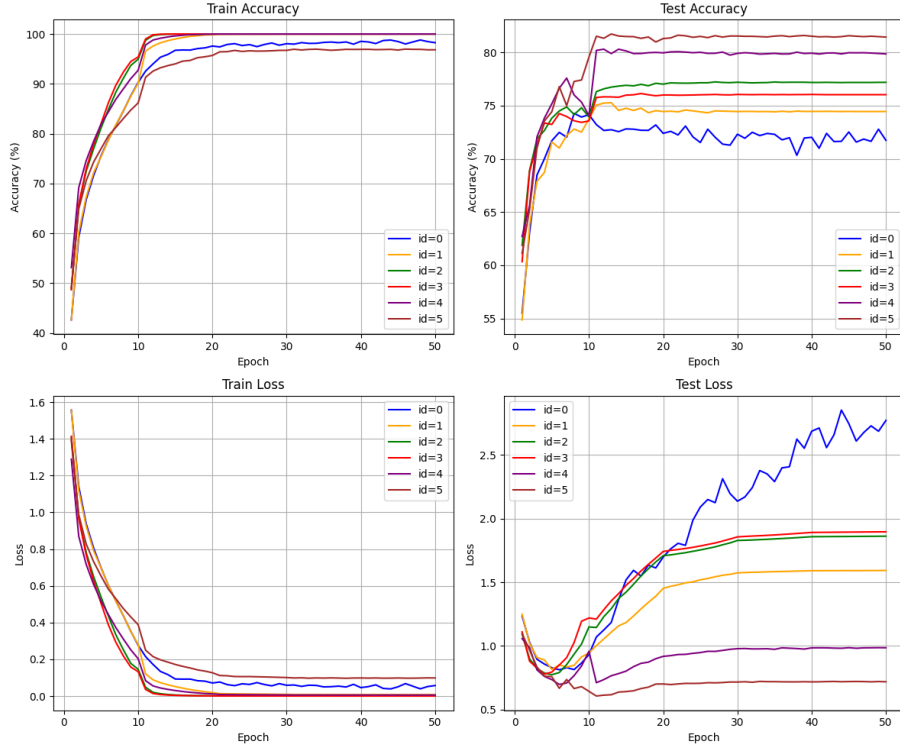


Figure 14: Train/test accuracy and loss for all models with progressively added methods

standardization, data augmentation with random cropping and horizontal flipping, batch normalization, and dropout.

Originally, we also considered adding ZCA whitening to further improve accuracy. However, the training time with ZCA was extremely long, and even though it could provide a small gain in test performance, the increase in computational cost was not justified. Therefore, we chose not to include ZCA whitening in our final models.

The graph below shows the different performances of the models we trained.

Model 0 does not show signs of overfitting, as its training accuracy remains slightly below 100%. Interestingly, Model 5 also avoids overfitting and even generalizes better than Model 0. Although all models display similar training accuracy trends, the test accuracy clearly distinguishes them: Model 5 achieves the highest performance, followed by Model 4. This improvement can be attributed to the combination of batch normalization and dropout, which together help the model learn more robust features while preventing overfitting.

Moreover, both Models 4 and 5 exhibit very stable loss curves throughout training, with Model 5 achieving the lowest overall loss, indicating more efficient and consistent optimization. Overall, according to our experiments, we can observe that the combination of multiple optimization, normalization, and regularization techniques, such as learning rate scheduling, standardization, data augmentation, batch normalization, and dropout leads to a model that trains efficiently, avoids overfitting, and achieves strong generalization performance on the test set.

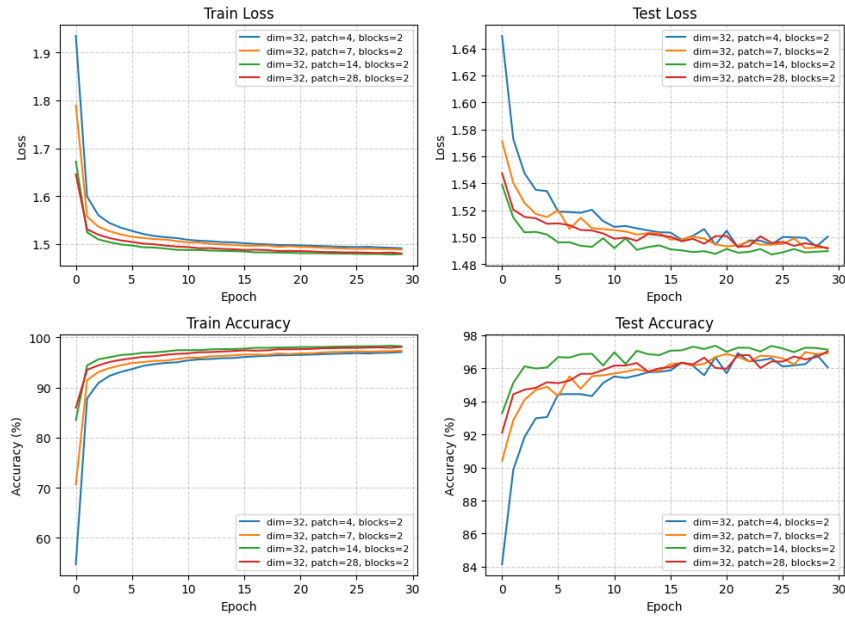


Figure 15: Grid searching the optimal patch size

Part III

e - Transformers

1 ViT model experiments

In this part, the Vision Transformer (ViT) model has been implemented from scratch using modular building blocks designed to replicate the architecture of the original Transformer while adapting it to image data. The implementation follows a layered design composed of four main classes: a feedforward MLP module, a `MultiHeadAttention` mechanism, a `Block` class that encapsulates normalization, attention, and MLP sublayers, and finally the global ViT model which assembles all these components into a trainable architecture.

Each image of shape $(B, 1, H, W)$ —with B the batch size, and $(H, W) = (28, 28)$ for MNIST—is first divided into non-overlapping patches of size $P \times P$, resulting in $\frac{H}{P} \times \frac{W}{P}$ patches per image. These patches are flattened and linearly projected into an embedding space of dimension D (`embed_dim`), producing a tensor of shape (B, N, D) , where N is the number of patches. A learnable class token is prepended to this sequence, and positional embeddings are added to preserve spatial information.

Each Transformer `Block` processes this sequence using multi-head self-attention. The attention mechanism splits the embedding dimension D across h heads (`num_heads`), computing queries, keys, and values of shape (B, h, N, D_h) , where $D_h = D/h$. The outputs of all heads are concatenated and projected back to dimension D . The MLP sublayer then applies a two-layer feedforward network.

The model stacks several such `Blocks` (`nb_blocks`) sequentially, progressively refining the patch representations through deeper contextual interactions. Finally, the class token’s embedding is passed to a classification head (a linear layer) that outputs class logits of shape (B, C) , where C is the number of classes (10 for MNIST).

1. **Test different hyperparameters and explain how they affect the performance. In particular `embed_dim`, `patch_size`, and `nb_blocks`.**

For this part, we modified the ViT class so that it can train and evaluate the model over different combinations of hyperparameters rather than fixed values. Specifically, I defined a set of possible values for `embed_dim`, `patch_size`, and `nb_blocks`, and our code loops through all of their combinations, training and testing the model for each configuration. The performance is evaluated by plotting both the loss and accuracy curves during training and

ViT — Grid Search on embedding dim

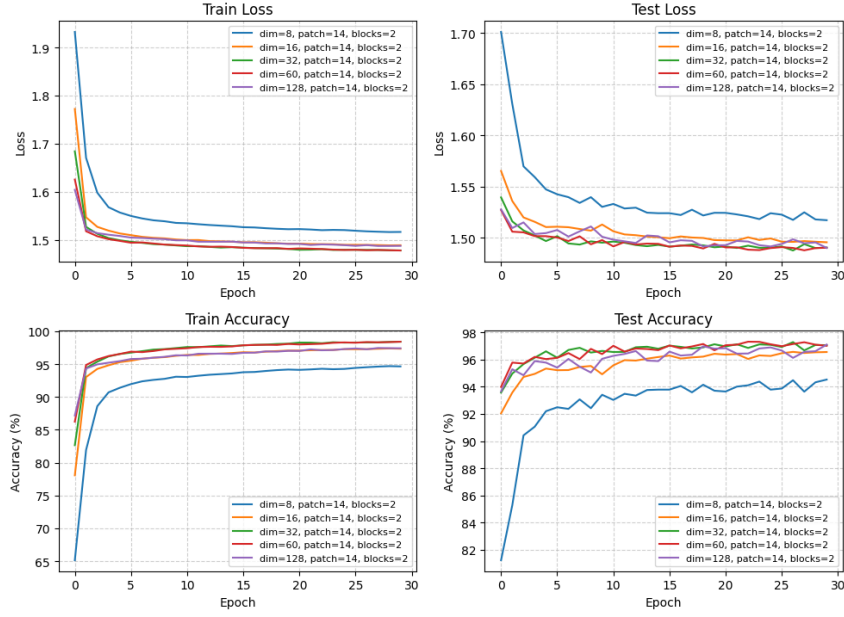


Figure 16: Grid searching the optimal embedding dimension

testing. To avoid large computation time, instead of performing a complete grid search over all three parameters simultaneously, we ran separate experiments in which we varied only one hyperparameter at a time while keeping the others fixed. This approach allows for a clearer observation of the impact of each parameter on the model’s performance. All experiments were run for 30 epochs.

Starting with the `patch_size`, we compared the default value of 7 with 4, 14, and 28, ensuring that each patch size was compatible with the input image dimensions of 28×28 . The results were all quite good and relatively similar as it shows in the figure 15, however, a patch size of 14 provided the best results, achieving a test accuracy of 0.971. A smaller patch size increases the number of patches per image, which enables the model to capture local details but also raises the computational cost and may lead to overfitting. Conversely, a larger patch size reduces the number of input tokens, simplifying the representation but causing a loss of spatial precision, which can degrade performance. We want a balance between locality and global context.

For the embedding dimension, we tested the default value of 32 along with lower values such as 8 and 16, and higher values such as 60 and 128. It is important to choose embedding dimensions that are divisible by the number of heads, which is set to 4 in this case by default. The patch size was kept fixed at 14 from the previous experiment. The figure 16 shows that a low embedding dimension, such as 8, does not allow the model to converge as effectively as higher ones, since it limits the representational capacity of each token and prevents the model from capturing sufficient information. Consequently, both training and testing accuracies stagnate around 0.91. As the embedding dimension increases, the model’s ability to encode richer and more discriminative features improves, resulting in significantly better performance with accuracies exceeding 0.96 for values of 16, 32, 60, and 128. However, beyond a certain threshold around 32–60, the gain becomes marginal, and the performance saturates. This suggests that higher embedding dimensions lead to diminishing returns while increasing computational cost therefore a risk of overfitting. The configuration with an embedding dimension of 32 thus provides the best balance between expressiveness, accuracy, and efficiency.

As for the number of blocks, we fixed the patch size to 14 and the embedding dimension to 32, and we experimented with different depths by setting hyperparameter to 2, 4, 8, and 16. Increasing the number of blocks directly increases the model’s depth and its ability to

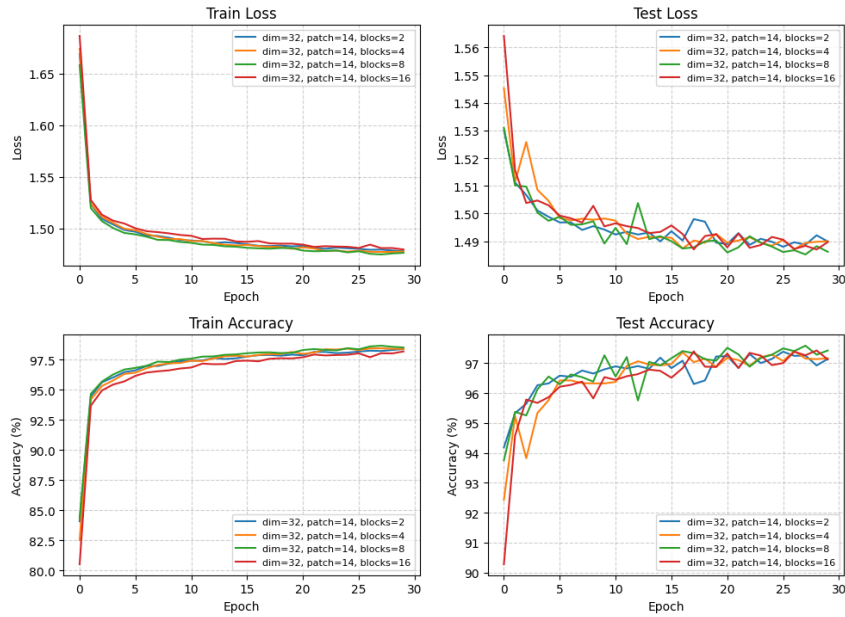


Figure 17: Grid searching the optimal number of blocks

build hierarchical feature representations. The results in figure 17 show that performance remains remarkably stable across configurations, with test accuracies oscillating between 97.1% and 97.4%, and training losses remaining around 1.48\$. While deeper models, such as with 8 or 16 blocks, achieve slightly higher peak accuracies, they also require considerably longer training times from approximately 10 seconds per epoch with 2 blocks to more than 24 seconds with 16 blocks. Moreover, the testing curves exhibit small oscillations around their convergence point, possibly due to a learning rate that is slightly too high for deeper architectures, whereas the training curves remain smooth and stable. Overall, these results suggest that increasing the number of blocks beyond a moderate depth yields only marginal gains in accuracy while substantially increasing computational cost and training instability. Therefore, a configuration with 2 or 4 blocks appears to offer the best trade-off between convergence speed, stability, and generalization performance.

2. Comment and discuss the final performance that you get. How to improve it?

The final performance of the Vision Transformer on the MNIST dataset is very satisfactory, reaching around 97% of test accuracy with stable and low loss values. This level of performance demonstrates that even with a limited number of blocks and moderate embedding dimension the model succeeds in capturing discriminative visual patterns from small grayscale images. However, several factors could still enhance its performance and generalization. First, data augmentation techniques such as random rotations, translations, or elastic deformations could increase the diversity of the training set and help the model become more robust to spatial variability. Second, regularization methods like dropout or weight decay could mitigate potential overfitting, especially when using deeper architectures. Third, a more systematic hyperparameter search might yield a better combination of patch size, embedding dimension, and learning rate. Finally, pretraining the transformer on a larger dataset or using transfer learning could provide richer feature representations and further boost performance on MNIST or similar datasets.

3. What is the complexity of the transformer in terms of number of tokens? How you can improve it?

The computational complexity of a Transformer primarily depends on the number of tokens N processed by the self-attention mechanism. Specifically, the standard self-attention operation exhibits a quadratic complexity $\mathcal{O}(N^2)$ in both time and memory, since it computes pairwise attention scores between all token pairs. In the case of a Vision Transformer, N corresponds

to the number of image patches, which scales inversely with the square of the patch size because

$$N = H * W / patch_size^2$$

where H and W are the height and width of the input images. Consequently, smaller patches lead to more tokens and thus higher computational cost.

To improve efficiency, several strategies can be considered. Increasing the patch size reduces the number of tokens and therefore the complexity, although this may slightly degrade fine-grained spatial information. Alternatively, efficient transformer variants such as the *Linearformer*, *Performers*, or *Swin Transformer* approximate or localize the attention mechanism, lowering the complexity to linear or near-linear with respect to N . Another way is hierarchical architectures that progressively reduce the number of tokens across layers, mimicking the multiscale structure of convolutional networks. These methods can substantially reduce memory and computation requirements while maintaining or even improving performance.

2 Large transformers

1. **Load the model using the `timm` library without pretrained weights. Try to apply it directly on a tensor with the same MNIST images resolution. What is the problem and why we have it? Explain if we have also such problem with CNNs. As ViT takes RGB images, the input tensor should have 3 channels.**

When applying this model directly to MNIST images, we encounter a dimensionality issue. The Vision Transformer (ViT) models implemented in `timm` are designed for ImageNet-like datasets, where images are RGB (3 channels) and have a fixed input size of 224×224 pixels. Since MNIST images are grayscale (1 channel) and smaller (28×28), the patch extraction layer in ViT cannot correctly divide the input image into non-overlapping patches of size 16×16 as expected. The model thus raises an error due to incompatible spatial dimensions. Even after replicating the single grayscale channel to obtain three channels ($3 \times 28 \times 28$), the input resolution is still insufficient: the ViT's patch embedding and positional encoding are initialized for a 224×224 resolution. As a result, the positional embedding shapes do not match, and the model cannot process the input unless it is resized to the expected dimension. This issue is specific to transformer-based architectures, which rely on fixed positional embeddings and expect a predefined number of patches corresponding to a particular input resolution. Conversely, CNNs are inherently more flexible: their convolutional filters operate locally and can process images of various sizes and aspect ratios. The only restriction in CNNs typically arises at the fully connected layers, where a fixed input size may be required but this can be manipulated by using adaptive pooling. Therefore, CNNs can usually handle 28×28 grayscale images directly, while ViTs require resizing and channel adaptation to match their predefined architecture.

2. **There is a trick in `timm` that allows to use pretrained models with different image resolution and number of classes.**

When directly applying a pretrained Vision Transformer to MNIST images (28×28), the model fails because its positional encodings and patch embeddings are defined for 224×224 inputs. By specifying the `img_size` argument when loading the model, the `timm` library automatically adapts the architecture to the new resolution by interpolating positional embeddings and resizing convolutional patch projections. This trick enables fine-tuning or inference on datasets with arbitrary resolutions without retraining the model from scratch.

3. **Comment the final results and provide some ideas on how to make transformer work on small datasets. You can take inspiration from some recent work.**

Two experimental configurations were evaluated to assess the performance of Vision Transformers (ViT) on the MNIST dataset: training from scratch and fine-tuning a pre-trained model. The ViT trained from scratch (`pretrained=False`) achieved a final training accuracy of approximately 92.5% and a test accuracy of 89.9% after 30 epochs. Convergence during the initial epochs was relatively slow, and a modest generalization gap (3 percentage points) was observed according to the figure 18, suggesting mild overfitting. These results are consistent with the known data-intensive nature of ViTs, which typically require large datasets to learn robust feature representations without prior knowledge.

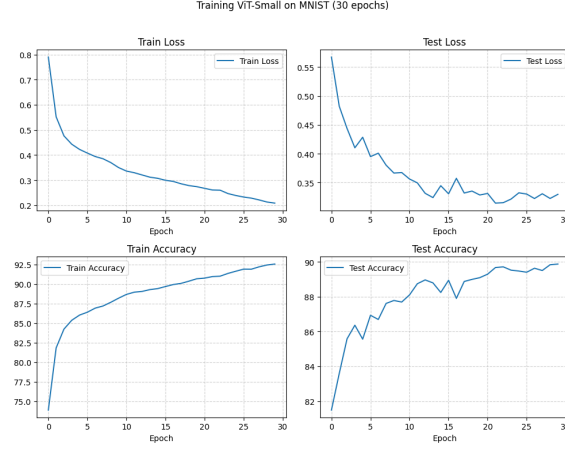


Figure 18: ViT-S model

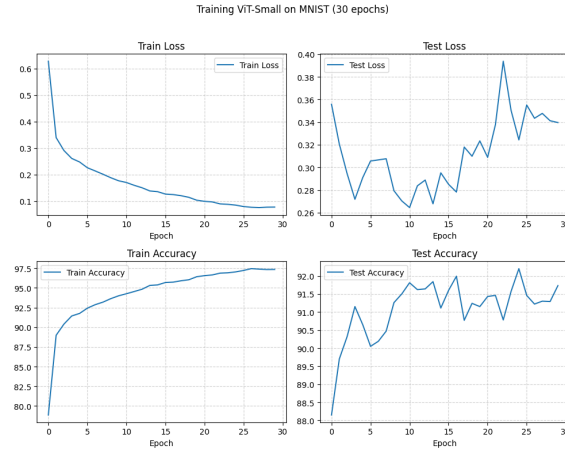


Figure 19: Pre-trained ViT-S model on ImageNet

In contrast, the ViT fine-tuned from a pre-trained checkpoint demonstrated improved performance, attaining 97.3% training accuracy and 91.7% test accuracy. Convergence was considerably faster, and the generalization gap was reduced as it's shown in the figure 19. These observations indicate that pre-trained representations, even when obtained from a domain such as ImageNet, provide strong inductive biases that are transferable to small-scale datasets, substantially mitigating the risk of overfitting.

Several strategies may further enhance transformer performance on limited datasets. Transfer learning and fine-tuning remain the most effective, as documented in recent studies like Training data-efficient image transformers & distillation through attention . Strong data augmentation, including geometric transformations, mixup, and cutout, can improve model generalization by artificially increasing data diversity. Knowledge distillation, as employed in DeiT, allows a smaller transformer to learn from a more capable teacher network. Additionally, architectural adaptations such as using smaller ViT variants, applying dropout, stochastic depth, and weight decay, or optimizing patch size for small images (e.g., 8×8 patches for MNIST) have proven beneficial. Finally, self-supervised pretraining approaches, such as masked autoencoders or contrastive learning, may provide a valuable pretext task to learn meaningful representations prior to supervised fine-tuning.

While ViTs are capable of competitive performance on MNIST, their efficiency and generalization on small datasets can be substantially improved through pre-training, architectural modifications, data augmentation, and self-supervised strategies. These results highlight the importance of leveraging prior knowledge and inductive biases when applying transformers in low-data regimes.